

Построение защищенных каналов связи

Макаров Артём

МИФИ 2025

Защищенный канал связи

- Обеспечить аутентичность и секретность сообщений при передаче по каналу связи при активном противнике
- Возможности противника – любые эффективные взаимодействия с каналом, включая повтор и модификацию передаваемых сообщений, возможность разовой компрометации сессионных ключей, возможность имперсонификации источника сообщений
- Цели противника – нарушение ИБ передаваемых сообщений

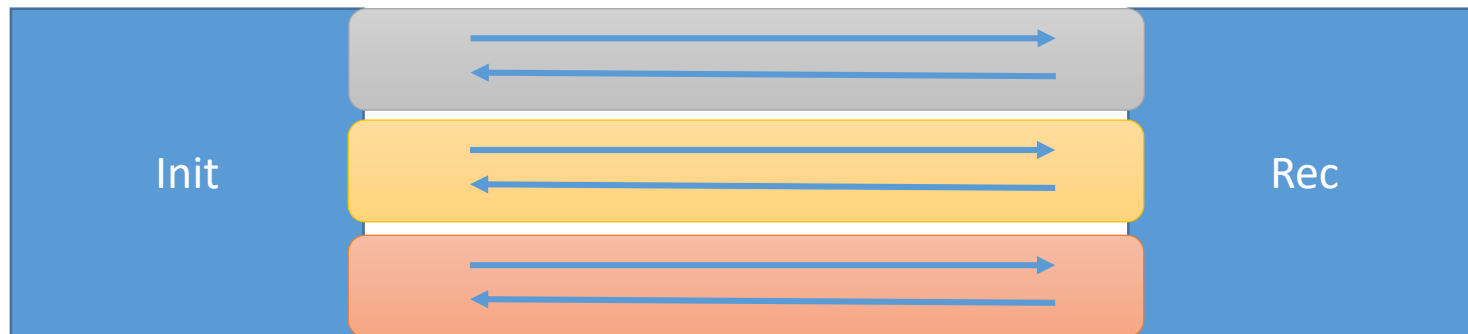
Защищенный канал связи

2 основных стороны – инициатор (Init) и получатель (Rec)

Нулевая фаза – согласование параметров

Первая фаза – согласование сессионного ключа

Вторая фаза – обмен сообщениями через защищенный канал

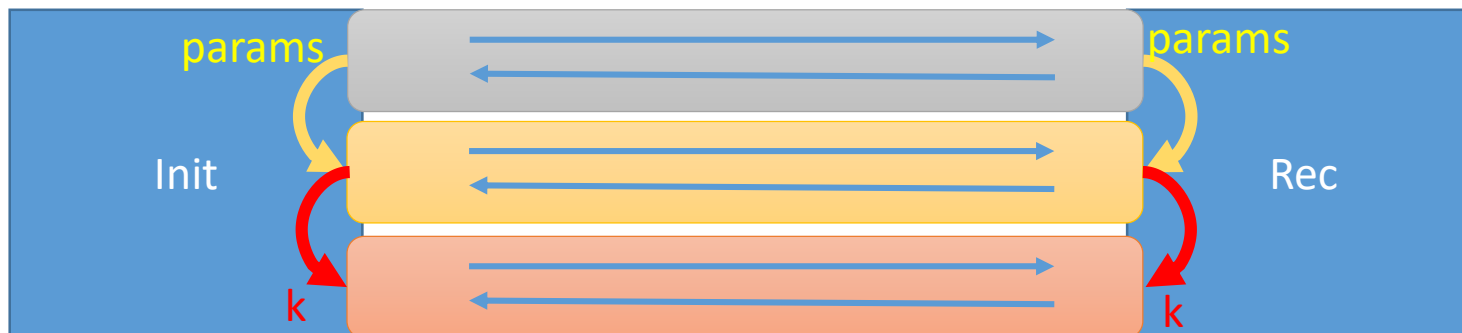


Защищенный канал связи

Нулевая фаза может отсутствовать, если параметры заранее известны сторонам. Результат – согласованные параметры на обеих сторонах.

Цель первой фазы выработка общего симметричного ключа, как правило асимметричным способом из общего ключевого материала (согласование ключа, Key establishment).

Третья фаза как правило осуществляется с помощью симметричной криптографии (AEAD).



Аутентификация в каналах связи

Очевидно, что для построения защищенного канала связи необходимо построение отношения доверия между сторонами (иначе стороны не будут знать, с кем они общаются).

Отношение доверия бывает односторонним и двухсторонним.

Одностороннее отношение доверия аутентифицирует только получателя.

Двухстороннее отношение доверия аутентифицирует и получателя и отправителя.

Аутентификация в каналах связи

Криптографическая аутентификация должна производиться на основе знания некоторого секрета, неизвестного противнику.

- Аутентифицируемая сторона должна иметь некоторый долгоживущий **секрет**
- Аутентифицирующая сторона должна иметь возможность **проверить** владение секретом
- Аутентифицирующая сторона должна иметь возможность сопоставить наличие секрета другой стороной с её **идентичностью**

Аутентификация с использованием асимметричной криптографии

- Секрет – **закрытый статический долговременный ключ**
- Проверка секрета – доказательство владения секретным ключом, **путём его использования**
- Привязка идентичности к **открытому ключу**

Возникает вопрос обмена открытыми ключами.

Решения:

- Использование удостоверяющего центра
- Прямой обмен ключами до протокольного взаимодействия

Некоторые требования к протоколам согласования ключей

- Аутентичность сущности (entity authentication) – кто то действует от лица конкретной сущности в ходе протокола
- Аутентичность источника (data origin authentication) – сообщения приходят от конкретного источника
- (неявная) Аутентичность ключа (implicit key authentication) – никто, кроме определённой сущности не может получить секретный ключ
- Подтверждение ключа (key confirmation) – возможность подтверждения какой-либо другой стороной факта владения выработанным секретным ключом
- Явная аутентичность ключа (explicit key authentication) – возможность подтверждения конкретной другой стороной факта обладания выработанным секретным ключом (неявная аутентичность + подтверждение ключа)

Некоторые модели стойкости протоколов

- Защита от чтения назад (forward secrecy, т.е. «дальнейшая стойкость») – компрометация **долговременных** ключей не приводит к компрометации прошлых **сессионных ключей**
- Защита от чтения вперёд (future secrecy, т.е. «будущая стойкость») – компрометация **сессионных ключей** не приводит к компрометации **будущих сессионных ключей**



- Rule #1 of protocol design: **Don't**
 - not even by simplifying existing protocols

TLS 1.3

RFC 8446

Цели

- Удаление legacy
- Увеличение производительности
- Уменьшение набора возможных атак

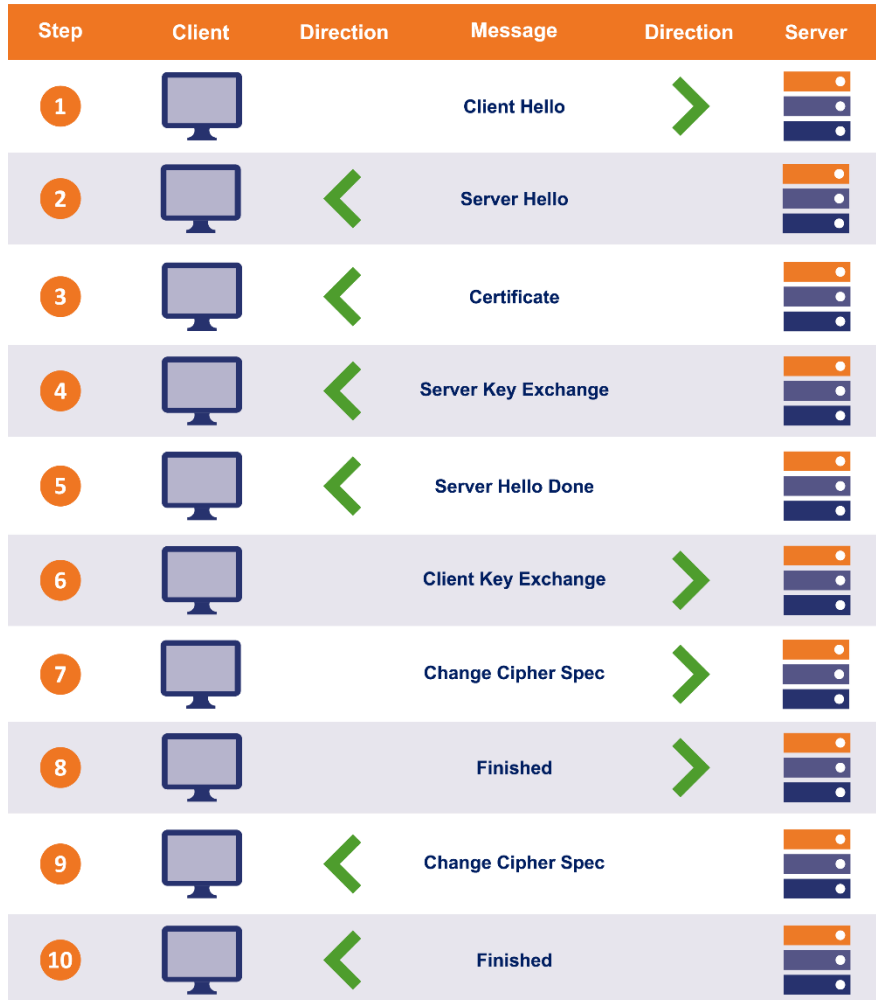
Согласование ключей в TLS 1.3

- Только эфемерный Диффи-Хеллман (удалили RSA)
- Только стойкий в настоящий момент набор параметров (определяющих группу).

Аутентифицированное шифрование

- Удаление проблемных и нестойких шифров (CBC, RC4)
- Разрешены только AEAD шифры

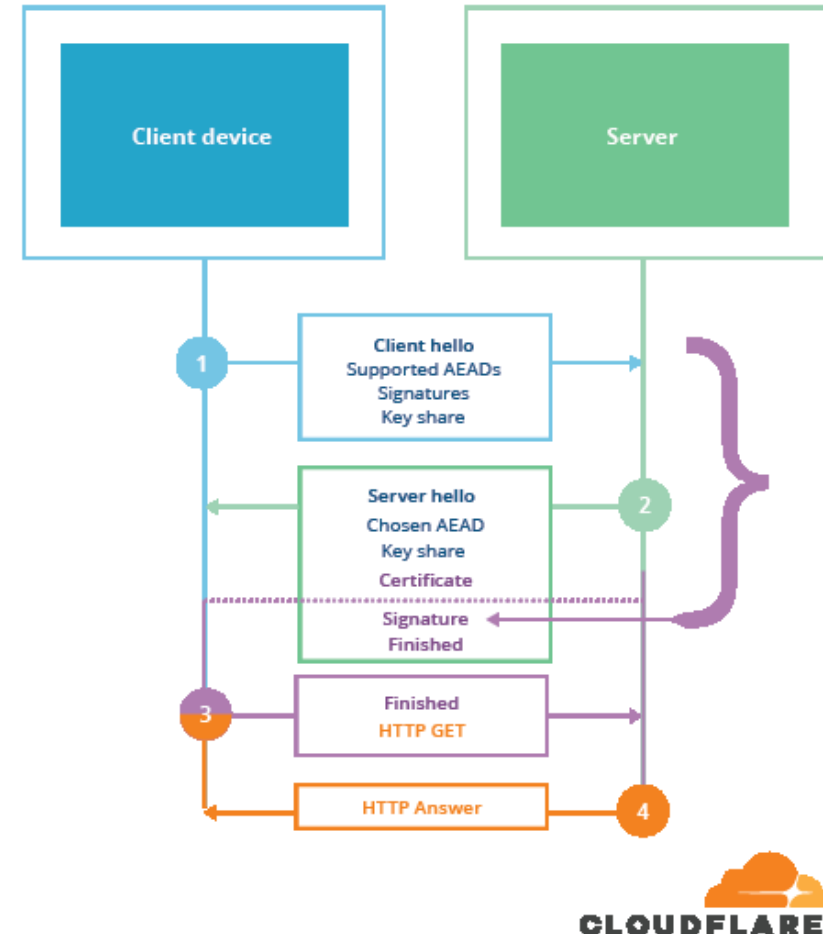
TLS 1.2 vs TLS 1.3 handshake



Электронные подписи

- Дополнение RSA-PSS (убрали PKCS#1 1.5)
- Сервер подписывает handshake (защищаясь от downgrade attack, когда in-the-middle противник навязывает сторонам нестойкие наборы)

TLS 1.3



Наборы алгоритмов

До версии 1.3 использовались наборы (cipher-suit) алгоритмов, определяющие используемые примитивы. Пример - ECDHE-ECDSA-AES-GCM-SHA256.

Версия 1.3 выделяет 3 независимых части названия

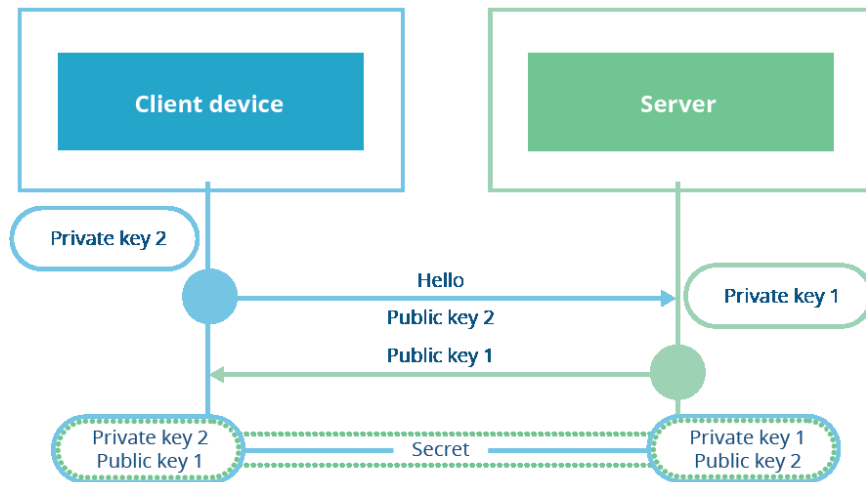
- Шифр + HKDF
- Протокол обмена ключами
- Алгоритм подписи

Пример – TLS_AES_128_GCM_SHA256, ECDHE, RSA_2048_SHA256

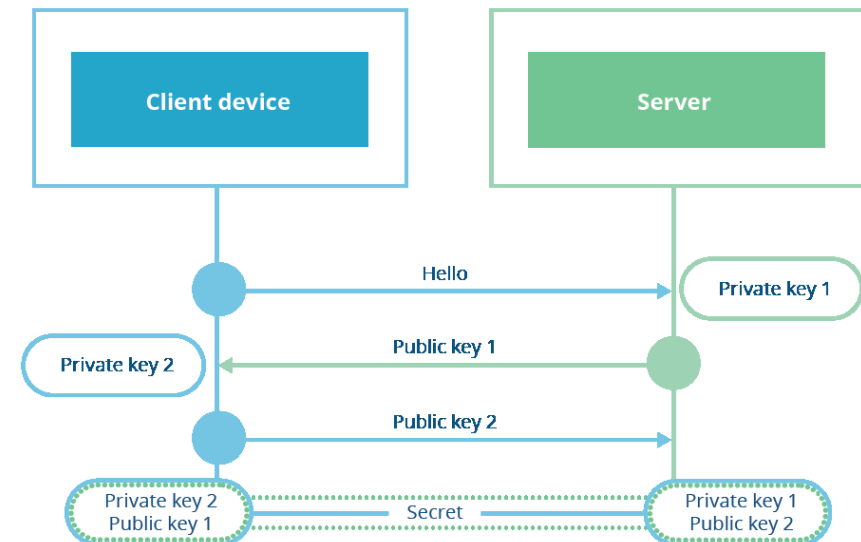
TLS 1.3 vs 1.2 public key Handshake

В 1.3 клиент может сразу отправить свой открытый ключ, не дожидаясь ответа от сервера со списком поддерживаемых алгоритмов (1-RTT).

DH 1.3 handshake



DH 1.2 handshake



TSL 1.3 ссылки

- <https://tls13.ulfheim.net/>
- <https://blog.cloudflare.com/rfc-8446-aka-tls-1-3/>
- <https://davidwong.fr/tls13/>
- Отображение suit'ов браузера
<https://clienttest.sslabs.com:8443/ssltest/viewMyClient.html>
- Краткий разбор пакетов 1.2 <https://medium.com/@ethicalevil/tls-handshake-protocol-overview-a39e8eee2cf5>

Noise

Каркасная модель построения протоколов (Protocol framework)

Основная идея – построения tls-образных протоколов с необходимым набором свойств.

Используемые примитивы – ECDH 25519/448 (Согласование ключей), SHA256/512 / BLAKE2b/BLAKE2s (hash, построение PRF в HKDF), AES-GCM/ChaChaPoly (AEAD).

Протоколы Noise

2 основных этапа – согласование ключа и использование ключа в защищенном канале связи

Согласование ключа – Диффи-Хеллман + HKDF

Защищенный канал – AEAD шифрование

Протоколы Noise

- Каждый протокол состоит из шаблонов
- шаблон описывает передаваемые сообщения в виде одного или нескольких токенов
- Каждая сторона протокола обрабатывает последовательность токенов. Сначала полностью происходит обработка на передающей стороне.
- После каждого обработки каждой строки в паттерне отправителем может быть передано сообщение, для защиты которого используется текущий симметричный ключ, при его наличии

Токены Noise

- "e" – сгенерировать и передать эфемерный ключ Диффи-Хеллмана / получить эфемерный ключ Диффи-Хеллмана
- "s" – передать статический ключ Диффи-Хеллмана / получить эфемерный ключ Диффи-Хеллмана
- "ee" – использовать свой эфемерный ключ и эфемерный ключ получателя для выработки симметричного ключа. Данный ключ используется для AEAD шифрование всех передаваемых в дальнейшем данных, пока не будет выработан новый ключ
- "es", "se", "ss", - аналогично "ee", но для комбинаций эфемерный-статический ключ. Первая буква в токене означает ключ передающей стороны.
- "psk" – использование пред-согласованного симметричного ключа для выработки сессионного ключа. Полученный ключ используется для AEAD шифрование всех передаваемых в дальнейшем данных, пока не будет выработан новый ключ

Пример Классический Диффи-Хеллман

- $\rightarrow e$
- $\leftarrow e, ee$

Строка 1.

Шаг 1. Генерация и отправка эфемерного ключа $Ae \rightarrow B$

Шаг 2. Сторона B получает эфемерный ключ Ae

Строка 2.

Шаг 3. Генерация и отправка эфемерного ключа $Be \rightarrow A$

Шаг 4. Выработка сессионного ключа из Ae и Be на стороне B

Шаг 5. Сторона A получает эфемерный ключ Bb

Шаг 6. Выработка сессионного ключа из Ae и Be на стороне A

Пример Диффи-Хеллман с односторонней и двухсторонней аутентификацией

- $\rightarrow e$
- $\leftarrow e, ee, s, es$
- $\rightarrow e$
- $\leftarrow e, ee, s, es$
- $\rightarrow s, se$

Именованние паттернов

Односторонние шаблоны реализуют неинтерактивное согласование ключа, при котором инициатор передаёт единственное сообщение

A():

<pre-messages>

...

<messages>

где A - имя шаблона, pre-message - сообщения, передаваемые в шаблоне пред-сообщения, message - сообщения передаваемые в шаблоне сообщения.

Именованние Шаблонов

- N: отсутствие статического ключа инициатора
- K: статический ключ инициатора известен получателю
- X: статический ключ инициатора передаётся получателю

Токены "s", "e", "e, s" внутри скобок означают что инициатор инициирован со следующими статическими / эфемерными ключами. Токены "rs", "re", "re", "rs" обозначают инициатор инициирован со знанием следующих открытых статических / эфемерных ключей получателя.

Пример:

N(rs):

<- s

...

-> e, es

Именованние шаблонов

Интерактивные шаблоны состоят из одного или нескольких шаблонов сообщений, передаваемых между получателем и инициатором.

AA():

<pre-messages>

...

<messages>

где AA - имя шаблона, pre-message - сообщения, передаваемые в шаблоне пред-сообщения, message - сообщения передаваемые в шаблоне.

Именованние шаблонов

Первая буква в имени шаблона определяет статический ключ инициатора, и может принимать одно из следующих значений:

- N: отсутствие статического ключа инициатора
- K: статический ключ инициатора известен получателю
- X: Статический ключ инициатора передаётся получателю
- I: Статический ключ инициатора немедленно передаётся получателю, даже в условиях отсутствия защиты идентичности.

Вторая буква в имени шаблона определяет статический ключ получателя, и может принимать одно из следующих значений:

- N: отсутствие статического ключа получателя
- K: статический ключ получателя известен инициатору
- X: статический ключ получателя передаётся инициатору.

Именованние паттернов

Пример

IK(s, rs):

<- s

...

-> e, es, s, ss

<- e, ee, se

Состояния Noise

Symmetric state

- h – текущее хэш-значение всех переданных и полученных данных
 - $h = \text{HASH}(h || \text{data})$
- ck – “chaining key”, используется как один из входов KDF (другой вход – результат DH)

Cipher state

- k – текущий ключ шифрования данных для AEAD
 - $(k, ck) = \text{HKDF}(DH(\dots), ck)$
 - Шифртекст $c = \text{AEAD}(\text{key: } k, \text{ad: } h, \text{plaintext})$
- n – nonce, выставляется 0 при смене k

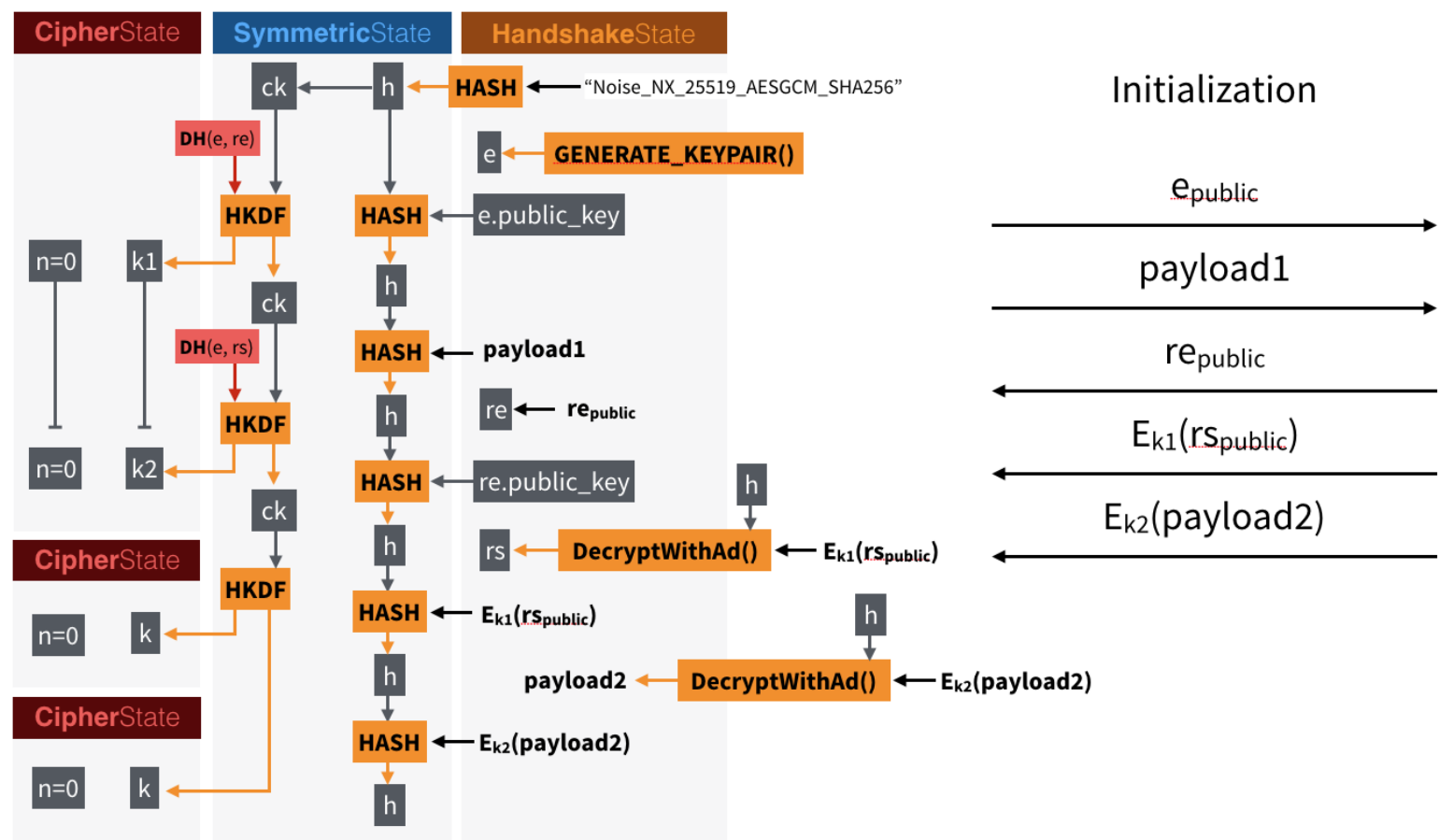
Функционирование протокола Noise

При каждом полученном или отправленном открытом ключе он подмешивается в h .

При отправке или получении любого открытого текста он подмешивается в h (после расшифрования, если он был зашифрован).

При выполнении DH вырабатывается новый ключ k , который используется для шифрования всех передаваемых в дальнейшем данных, включая открытые ключи.

Функционирование протокола Noise



Пример

NX:

-> e, payload1

<- e, ee, s, se, payload2

Свойства payload (инициатор)

0. No authentication. This payload may have been sent by any party, including an active attacker.

1. Sender authentication vulnerable to key-compromise impersonation (KCI). The sender authentication is based on a static-static DH ("ss") involving both parties' static key pairs. If the recipient's long-term private key has been compromised, this authentication can be forged. Note that a future version of Noise might include signatures, which could improve this security property, but brings other trade-offs.

2. Sender authentication resistant to key-compromise impersonation (KCI). The sender authentication is based on an ephemeral-static DH ("es" or "se") between the sender's static key pair and the recipient's ephemeral key pair. Assuming the corresponding private keys are secure, this authentication cannot be forged

Свойства payload (получатель)

0. No confidentiality. This payload is sent in cleartext.

1. Encryption to an ephemeral recipient. This payload has forward secrecy, since encryption involves an ephemeral-ephemeral DH ("ee"). However, the sender has not authenticated the recipient, so this payload might be sent to any party, including an active attacker.

2. Encryption to a known recipient, forward secrecy for sender compromise only, vulnerable to replay. This payload is encrypted based only on DHs involving the recipient's static key pair. If the recipient's static private key is compromised, even at a later date, this payload can be decrypted. This message can also be replayed, since there's no ephemeral contribution from the recipient.

Свойства payload (получатель)

3. Encryption to a known recipient, weak forward secrecy. This payload is encrypted based on an ephemeral-ephemeral DH and also an ephemeral-static DH involving the recipient's static key pair. However, the binding between the recipient's alleged ephemeral public key and the recipient's static public key hasn't been verified by the sender, so the recipient's alleged ephemeral public key may have been forged by an active attacker. In this case, the attacker could later compromise the recipient's static private key to decrypt the payload.

Свойства payload (получатель)

4. Encryption to a known recipient, weak forward secrecy if the sender's private key has been compromised. This payload is encrypted based on an ephemeral-ephemeral DH, and also based on an ephemeral-static DH involving the recipient's static key pair. However, the binding between the recipient's alleged ephemeral public and the recipient's static public key has only been verified based on DHs involving both those public keys and the sender's static private key. Thus, if the sender's static private key was previously compromised, the recipient's alleged ephemeral public key may have been forged by an active attacker. In this case, the attacker could later compromise the intended recipient's static private key to decrypt the payload (this is a variant of a "KCI" attack enabling a "weak forward secrecy" attack). Note that a future version of Noise might include signatures, which could improve this security property, but brings other trade-offs.

Свойства payload (получатель)

5. Encryption to a known recipient, strong forward secrecy. This payload is encrypted based on an ephemeral-ephemeral DH as well as an ephemeral-static DH with the recipient's static key pair. Assuming the ephemeral private keys are secure, and the recipient is not being actively impersonated by an attacker that has stolen its static private key, this payload cannot be decrypted.

Больше про noise

- <https://noiseprotocol.org/>
- <https://www.nccgroup.com/us/about-us/newsroom-and-events/blog/2018/august/introducing-disco/>

WireGuard

Черновик стандарта построения VPN соединений.

- Все пакеты аутентифицированы, неаутентифицированные отбрасываются. Не отвечать на неаутентифицированные пакеты.
- 1 RRT handshake (Noise IK)
- Использование метки времени TAI64N в первом сообщении
- Опциональное использование psk для смягчения угрозы перед квантовым противником
- Использование cookie для смягчения DOS

Маршрутизация WireGuard

Сетевой интерфейс имеет статическую пару ключей, назначенный порт и таблицу маршрутизации. В таблице маршрутизации для каждой записи хранится статический открытый ключ принимающей стороны.

Configuration 1a

Interface Public Key	Interface Private Key	Listening UDP Port
HIgo...8ykw	yAnz...fBmk	41414
Peer Public Key	Allowed Source IPs	
xTIB...p8Dg	10.192.122.3/32, 10.192.124.0/24	
TrMv...WXX0	10.192.122.4/32, 192.168.0.0/16	
gN65...z6EA	10.10.10.230/32	

Маршрутизация WireGuard

При отправке или получении пакета через интерфейс WireGuard из таблицы маршрутизации извлекается соответствующий открытый ключ, используемый для построения защищенного канала связи.

При получении пакета от других узлов их адреса добавляются в таблицу маршрутизации в поле Internet Endpoint. Данное поле может быть заполнено вручную на этапе конфигурации.

Пусть узел с конфигурацией 2b отправляет пакет узлу с предконфигурированным адресом 192.95.5.69:41414. Тогда на узле получателя при получении пакета будет добавлена запись Internet Endpoint об узле отправителя (конфигурация 1b). Порты отправки и получения должны совпадать.

Маршрутизация WireGuard

Configuration 2b

Interface Public Key gN65...z6EA	Interface Private Key gI6E...fWGE	Listening UDP Port 21841
Peer Public Key HIgo...8ykw	Allowed Source IPs 0.0.0.0/0	Internet Endpoint 192.95.5.69:41414

Configuration 1b

Interface Public Key HIgo...8ykw	Interface Private Key yAnz...fBmk	Listening UDP Port 41414
Peer Public Key xTIB...p8Dg TrMv...WXX0 gN65...z6EA	Allowed Source IPs 10.192.122.3/32, 10.192.124.0/24 10.192.122.4/32, 192.168.0.0/16 10.10.10.230/32	Internet Endpoint 192.95.5.64:21841

Процедура инкапсуляции WireGuard

Пакет с ОТ приходит на интерфейс WireGuard.

Определяется узел получателя и его открытый ключ.

Определяется ключ и счётчик для использования в ChaCha20Poly1305. Содержимое пакета шифруется.

К содержимому пакета добавляется заголовок.

Результат отправляется узлу в виде UDP пакета по адресу Internet Endpoint.

Криптография WireGuard

Основа – Noise «IK» (явная аутентичность ключа + forward secrecy)

IK(s, rs):

$\leftarrow s$

...

$\rightarrow e, es, s, ss$

$\leftarrow e, ee, se$

Первое передаваемое сообщение после handshake аутентифицирует инициатора.

Используемые примитивы – ECDH 25519, ChaCha20Poly1305, HKDF, BLAKE2s (MAC, PRF), BLAKE2s (Hash), HMAC

Использование метки времени

Благодаря 1-RTT нет необходимости хранить состояние при согласовании ключа.

Возможна атака повтором, в которой противник повторяет handshake, заставляя получателя разорвать существующую сессию (DOS).

Для защиты используется метка времени в формате TAI64N, используемая в теле первого передаваемого инициатором сообщения. Получатель хранит последнюю полученную метку времени. Все метки времени, младше полученной считаются недействительными.

Использование psk

Стойкость протокола основана на стойкости ECDH, которая сводится к сложности задачи дискретного логарифмирования.

При квантовом противники данная задача является эффективно вычислимой.

Возможна квантовая атака – противник хранит все текущие сессии, до момента получения в своё распоряжение квантового компьютера. Затем он может расшифровать весь перехваченный трафик, построив атаку на ECDH.

Возможно использование смягчения – использования статического симметричного ключа psk, который используется для «подмешивания» в sk symmetric state (см токен psk Noise) как начальный ключевой материал HKDF.

Cookie и DOS

Возможна атака на узел получателя в виде отправки множества сообщений в рамках handshake. Для ответов ему необходимо вычислять ECDH, что является трудозатратой операций и может привести к DOS.

Смягчение – при нагрузке узел может не отвечать на handshake а отправлять специальный ответ с cookie, ожидая особый ответ от инициатора.



Cookie и DOS

Получатель (целевой узел) хранит секретное случайное значение s , изменяемое каждые 2 минуты. Cookie – есть MAC от IP инициатора, с использованием секретного значения в качестве ключа ($c = MAC(s, IP)$).

Инициатор при получении cookie использует его в качестве ключа для вычисления MAC от собственного запроса $c_r = MAC(c, request)$.

Получатель проверяет MAC c_r от запроса пользователя.

Цель – привязка источника сообщений к его адресу. Количество сообщений с фиксированного IP можно ограничивать стандартными средствами.

Cookie и DOS

Возможны проблемы – cookie не защищены, приходится отвечать на неаутентифицированные сообщения, противник может организовать DOS инициатора, отправляя невалидные cookie

Решение – больше cookie богу cookie

Использование двух различных MAC в качестве ответов на cookie-запросы



... приходится отвечать на
неаутентифицированные сообщения

msg.mac1 – используем хэш от
открытого ключа получателя в качестве
ключа и вычисляем для всех байтов
сообщения до msg.mac1. Присутствует
в сообщении инициатора при
handshake всегда, даже без нагрузки.

Цель – инициатор должен знать с кем
он общается и какой у него РК (нет атак
«из вне» от сторонних систем).



... cookie не защищены

Cookie будут шифроваться AEAD с использованием открытого ключа получателя. Цель – не знаешь PK получателя, не можешь пользоваться cookie.



... DOS на инициатора

Отвечая зашифрованной AEAD cookie-запрос инициатор добавляет в присоединённые данные (AD) msg.mac1 исходного сообщения инициатора. Цель – инициатор уверен, что cookie создана для его запроса.



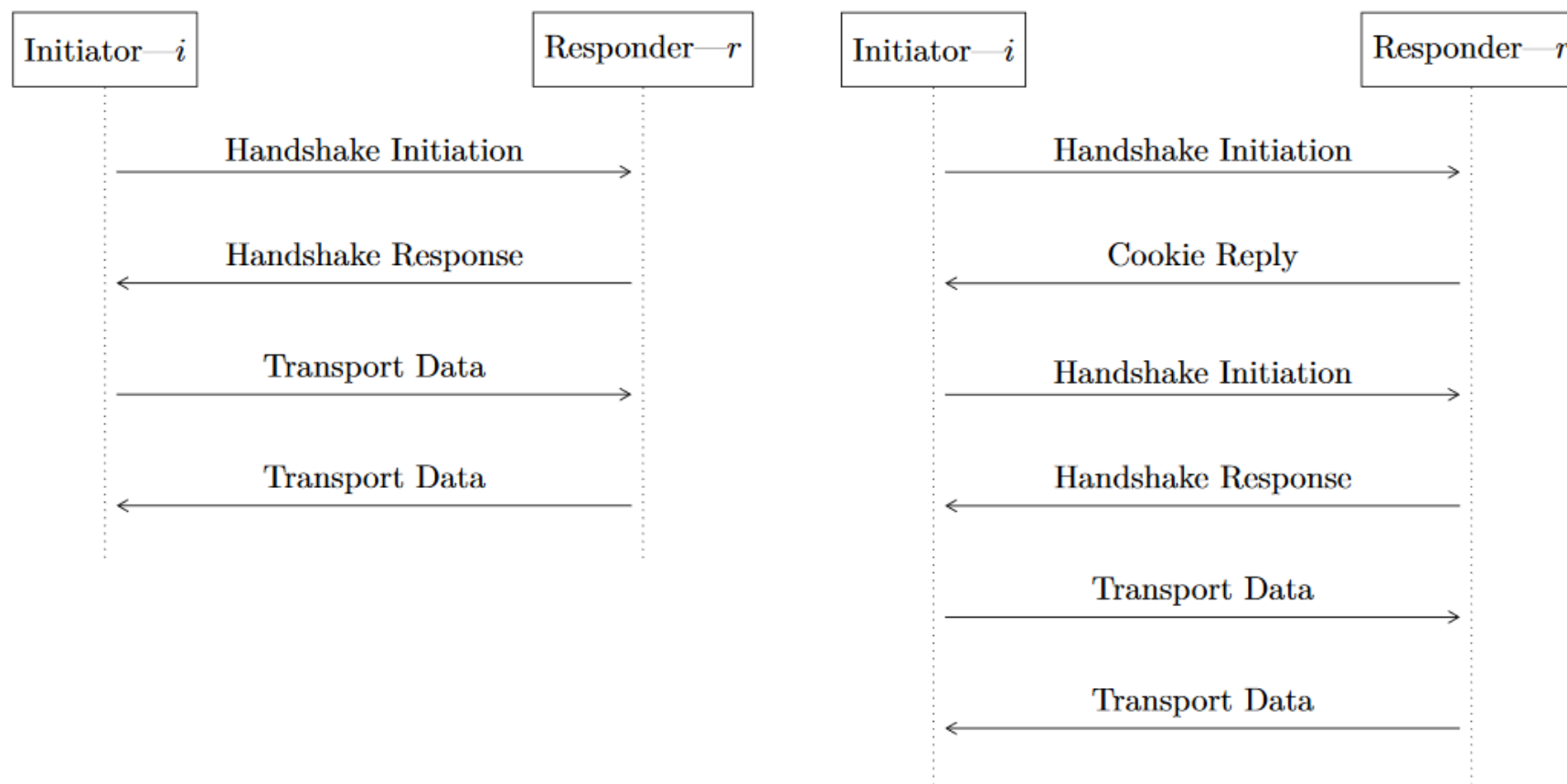
Второй MAC

msg.mac2 использует cookie в качестве ключа и вычисляется для всех байтов сообщения до msg.mac2. Получатель проверяет данный MAC и отвечает на handshake. Используется только под нагрузкой.

Итого – получатель всегда проверяет msg.mac1 в первом сообщении, если под нагрузкой отправляет зашифрованную cookie с msg.mac1 в качестве AD, затем проверяет msg.mac2 и продолжает handshake. Если нагрузки нет - msg.mac2 не проверяется.

msg.mac1 и msg.mac2 аутентифицируют все передаваемые данные в сообщении на момент их формирования.

Итого



Сообщение инициатора

type := 0x1 (1 byte)	reserved := 0 ³ (3 bytes)
sender := I_i (4 bytes)	
ephemeral (32 bytes)	
static ($\widehat{32}$ bytes)	
timestamp ($\widehat{12}$ bytes)	
mac1 (16 bytes)	mac2 (16 bytes)

$$\begin{aligned}
 C_i &:= \text{HASH}(\text{CONSTRUCTION}) \\
 H_i &:= \text{HASH}(C_i \parallel \text{IDENTIFIER}) \\
 H_i &:= \text{HASH}(H_i \parallel S_r^{\text{pub}}) \\
 (E_i^{\text{priv}}, E_i^{\text{pub}}) &:= \text{DH-GENERATE}() \\
 C_i &:= \text{KDF}_1(C_i, E_i^{\text{pub}}) \\
 \text{msg.ephemeral} &:= E_i^{\text{pub}} \\
 H_i &:= \text{HASH}(H_i \parallel \text{msg.ephemeral}) \\
 (C_i, \kappa) &:= \text{KDF}_2(C_i, \text{DH}(E_i^{\text{priv}}, S_r^{\text{pub}})) \\
 \text{msg.static} &:= \text{AEAD}(\kappa, 0, S_i^{\text{pub}}, H_i) \\
 H_i &:= \text{HASH}(H_i \parallel \text{msg.static}) \\
 (C_i, \kappa) &:= \text{KDF}_2(C_i, \text{DH}(S_i^{\text{priv}}, S_r^{\text{pub}})) \\
 \text{msg.timestamp} &:= \text{AEAD}(\kappa, 0, \text{TIMESTAMP}(), H_i) \\
 H_i &:= \text{HASH}(H_i \parallel \text{msg.timestamp}) \\
 \\
 \text{msg.mac1} &:= \text{MAC}(\text{HASH}(\text{LABEL-MAC1} \parallel S_{m'}^{\text{pub}}), \text{msg}_\alpha) \\
 \text{if } L_m = \epsilon \text{ or } \widetilde{L}_m \geq 120: \\
 \quad \text{msg.mac2} &:= 0^{16} \\
 \text{otherwise:} \\
 \quad \text{msg.mac2} &:= \text{MAC}(L_m, \text{msg}_\beta)
 \end{aligned}$$

L_m – последняя полученная инициатором cookie
 $\widetilde{L}_m$ – время с момента получения последней cookie инициатором (сек)

Сообщение получателя

type := 0x2 (1 byte)	reserved := 0 ³ (3 bytes)
sender := I_r (4 bytes)	receiver := I_i (4 bytes)
ephemeral (32 bytes)	
empty ($\hat{0}$ bytes)	
mac1 (16 bytes)	mac2 (16 bytes)

$(E_r^{priv}, E_r^{pub}) := \text{DH-GENERATE}()$

$C_r := \text{KDF}_1(C_r, E_r^{pub})$

$\text{msg.ephemeral} := E_r^{pub}$

$H_r := \text{HASH}(H_r \parallel \text{msg.ephemeral})$

$C_r := \text{KDF}_1(C_r, \text{DH}(E_r^{priv}, E_i^{pub}))$

$C_r := \text{KDF}_1(C_r, \text{DH}(E_r^{priv}, S_i^{pub}))$

$(C_r, \tau, \kappa) := \text{KDF}_3(C_r, Q)$

$H_r := \text{HASH}(H_r \parallel \tau)$

$\text{msg.empty} := \text{AEAD}(\kappa, 0, \epsilon, H_r)$

$H_r := \text{HASH}(H_r \parallel \text{msg.empty})$

$\text{msg.mac1} := \text{MAC}(\text{HASH}(\text{LABEL-MAC1} \parallel S_{m'}^{pub}), \text{msg}_\alpha)$

if $L_m = \epsilon$ or $\widetilde{L}_m \geq 120$:

$\text{msg.mac2} := 0^{16}$

otherwise:

$\text{msg.mac2} := \text{MAC}(L_m, \text{msg}_\beta)$

L_m – последняя полученная инициатором cookie

$\widetilde{L}_m$ – время с момента получения последней cookie инициатором (сек)

Сообщение с cookie

type := 0x3 (1 byte)	reserved := 0 ³ (3 bytes)
receiver := $I_{m'}$ (4 bytes)	
nonce := ρ^{24} (24 bytes)	
cookie ($\hat{16}$ bytes)	

The secret variable, R_m , changes every two minutes to a random value, $A_{m'}$ represents a concatenation of the subscript's external IP source address and UDP source port, and M represents the msg.mac1 value of the message to which this is in reply. The remaining encrypted cookie reply field is populated as such:

$$\tau := \text{MAC}(R_m, A_{m'})$$

$$\text{msg.cookie} := \text{XAEAD}(\text{HASH}(\text{LABEL-COOKIE} \parallel S_m^{\text{pub}}), \text{msg.nonce}, \tau, M)$$

Выработка транспортного ключа

- Нулевые начальные nonce, KDF от ck (см Noise).

$$(T_i^{send} = T_r^{recv}, T_i^{recv} = T_r^{send}) := \text{KDF}_2(C_i = C_r, \epsilon)$$

$$N_i^{send} = N_r^{recv} = N_i^{recv} = N_r^{send} := 0$$

$$E_i^{priv} = E_i^{pub} = E_r^{priv} = E_r^{pub} = C_i = C_r := \epsilon$$

T_i, T_r – ключи инициатора и получателя

N_i, N_r – nonce

E_i – ключевая пара (зануляются)

C_i – результаты вычисления KDF (symmetric state ck, зануляются)

Использование транспортного ключа

type := 0x4 (1 byte)	reserved := 0 ³ (3 bytes)
receiver := $I_{m'}$ (4 bytes)	
counter (8 bytes)	
packet ($\widehat{\ P\ }$ bytes)	

$$P := P \parallel 0^{16 \cdot \lceil \|P\|/16 \rceil - \|P\|}$$

$$\text{msg.counter} := N_m^{\text{send}}$$

$$\text{msg.packet} := \text{AEAD}(T_m^{\text{send}}, N_m^{\text{send}}, P, \epsilon)$$

$$N_m^{\text{send}} := N_m^{\text{send}} + 1$$

WireGuard

- Активно внедряется с 2020 года
- Whitepaper <https://www.wireguard.com/papers/wireguard.pdf>